Zip Code Group Project 1.0

1.0

Generated by Doxygen 1.18.0

1 Class Index 1

1.1 Class List . . . 1

2 File Index 3

2.1 File List . . . 3

3 Class Documentation 5

3.1 Buffer Class Reference . . . 5

3.1.1 Detailed Description . . . 6

3.1.2 Constructor & Destructor Documentation . . . 6

3.1.2.1 Buffer() . . . 6

3.1.3 Member Function Documentation . . . 7

3.1.3.1 displayTable() . . . 7

3.1.3.2 findExtrema() . . . 7

3.1.3.3 getEasternmostZipCode() . . . 8

3.1.3.4 getLatitude() . . . 9

3.1.3.5 getLongitude() . . . 9

3.1.3.6 getNorthernmostZipCode() . . . 10

3.1.3.7 getPlaceName() . . . 10

3.1.3.8 getSouthernmostZipCode() . . . 11

3.1.3.9 getState() . . . 11

3.1.3.10 getWesternmostZipCode() . . . 12

3.1.3.11 getZipCode() . . . 12

3.1.3.12 readCSV() . . . 13

3.1.3.13 setLatitude() . . . 14

3.1.3.14 setLongitude() . . . 14

3.1.3.15 setPlaceName() . . . 15

3.1.3.16 setState() . . . 15

3.1.3.17 setZipCode() . . . 16

3.1.4 Member Data Documentation . . . 16

3.1.4.1 extremes . . . 16

3.1.4.2 latitude . . . 16

3.1.4.3 longitude . . . 16

3.1.4.4 placeName . . . 17

3.1.4.5 state . . . 17

3.1.4.6 zipCode . . . 17

3.2 Extremes Struct Reference . . . 17

3.2.1 Detailed Description . . . 18

3.2.2 Member Data Documentation . . . 18

3.2.2.1 east . . . 18

3.2.2.2 eastLongitude . . . 18
3.2.2.3 north . . . 19
3.2.2.4 northLatitude . . . 19
3.2.2.5 south . . . 19
3.2.2.6 southLatitude . . . 19
3.2.2.7 west . . . 19
3.2.2.8 westLongitude . . . 19

4 File Documentation **21**

4.1 main.cpp File Reference . . . 21
4.1.1 Detailed Description . . . 21
4.1.2 Function Documentation . . . 21
4.1.2.1 main() . . . 21
4.2 main.cpp . . . 22
4.3 ZipCodeBuffer.cpp File Reference . . . 22
4.3.1 Detailed Description . . . 23
4.4 ZipCodeBuffer.cpp . . . 23
4.5 ZipCodeBuffer.h File Reference . . . 26
4.5.1 Detailed Description . . . 26
4.6 ZipCodeBuffer.h . . . 26

Index **29**

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Buffer
: Reads records sequentially and accumulates geographic extrema 5

Extremes
: ZIP codes and comparison coordinates for one state 17

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

main.cpp
    Application entry point for the ZIP code group project 21
ZipCodeBuffer.cpp
    Implements CSV parsing, extrema accumulation, and table output 22
ZipCodeBuffer.h
    Record buffer and per-state geographic extrema declarations 26

Chapter 3

Class Documentation

3.1 Buffer Class Reference

Reads records sequentially and accumulates geographic extrema.

`#include <ZipCodeBuffer.h>`

Public Member Functions

- Buffer ()
  Initializes an empty record and empty extrema map.
- void setState (string state)
  Sets the current record State.
- void setPlaceName (string placeName)
  Sets the current record PlaceName.
- void setLatitude (double latitude)
  Sets the current record Latitude.
- void setLongitude (double longitude)
  Sets the current record Longitude.
- void setZipCode (int zipCode)
  Sets the current record ZipCode.
- string getState ()
  Returns State.
- string getPlaceName ()
  Returns PlaceName.
- double getLatitude ()
  Returns Latitude.
- double getLongitude ()
  Returns Longitude.
- int getZipCode ()
  Returns ZipCode.
- bool readCSV (string fileName)

Reads the CSV and accumulates one record at a time.

- void findExtrema ()

  Updates state extrema using the current buffered record.
- int getEasternmostZipCode (string state)

  Returns EasternmostZipCode.
- int getWesternmostZipCode (string state)

  Returns WesternmostZipCode.
- int getNorthernmostZipCode (string state)

  Returns NorthernmostZipCode.
- int getSouthernmostZipCode (string state)

  Returns SouthernmostZipCode.
- void displayTable ()

  Prints the state extrema table to standard output.

Private Attributes

- string state

  Current record state.
- string placeName

  Current record placeName.
- double latitude

  Current record latitude.
- double longitude

  Current record longitude.
- int zipCode

  Current record zipCode.
- map< string, Extremes > extremes

3.1.1 Detailed Description

Reads records sequentially and accumulates geographic extrema.

Stores the current record and a state-keyed map. Equal coordinates select the smaller ZIP code, making results independent of record order.

Definition at line 29 of file ZipCodeBuffer.h.

3.1.2 Constructor & Destructor Documentation

3.1.2.1 Buffer()

```
Buffer::Buffer ()
```

Initializes an empty record and empty extrema map.

Definition at line 10 of file ZipCodeBuffer.cpp.

```
00010                {
00011     state = "";
00012     placeName = "";
00013     latitude = 0.0;
00014     longitude = 0.0;
00015     zipCode = 0;
00016 }
```

References latitude, longitude, placeName, state, and zipCode.

3.1.3 Member Function Documentation

3.1.3.1 displayTable()

```
void Buffer::displayTable ()
```

Prints the state extrema table to standard output.

Rows follow alphabetical state order. Columns are state, east, west, north, south. ZIP codes are padded to five digits.

Definition at line 195 of file ZipCodeBuffer.cpp.

```
00195                           {
00196     cout « left;
00197     cout « setw(7) « "State";
00198     cout « setw(14) « "Easternmost";
00199     cout « setw(14) « "Westernmost";
00200     cout « setw(14) « "Northernmost";
00201     cout « setw(14) « "Southernmost" « endl;
00202     cout « "-------------------------------------------------------------" « endl;
00203
00204     // Go through every state in the map
00205     for(pair<string, Extremes> entry : extremes){
00206         string state = entry.first;
00207
00208         cout « left « setfill(' ') « setw(7) « state;
00209
00210         // setw(5) and setfill('0') print Zip Codes like 501 as 00501
00211         cout « right « setfill('0');
00212         cout « setw(5) « getEasternmostZipCode(state) « "          ";
00213         cout « setw(5) « getWesternmostZipCode(state) « "          ";
00214         cout « setw(5) « getNorthernmostZipCode(state) « "          ";
00215         cout « setw(5) « getSouthernmostZipCode(state) « endl;
00216     }
00217
00218     cout « setfill(' ');
00219 }
```

References extremes, getEasternmostZipCode(), getNorthernmostZipCode(), getSouthernmostZipCode(), getWesternmostZipCode(), and state.

Referenced by main().

3.1.3.2 findExtrema()

```
void Buffer::findExtrema ()
```

Updates state extrema using the current buffered record.

East uses maximum longitude, west minimum longitude, north maximum latitude, and south minimum latitude. Ties use the smaller ZIP.

Definition at line 114 of file ZipCodeBuffer.cpp.

```
00114                           {
00115
00116     // First time seeing this state, so this Zip Code is the farthest in every direction
00117     if(extremes.count(state) == 0){
00118         Extremes extreme;
00119         extreme.east = zipCode;
00120         extreme.west = zipCode;
00121         extreme.north = zipCode;
00122         extreme.south = zipCode;
00123         extreme.eastLongitude = longitude;
00124         extreme.westLongitude = longitude;
```

```
00125         extreme.northLatitude = latitude;
00126         extreme.southLatitude = latitude;
00127 
00128         extremes[state] = extreme;
00129         return;
00130     }
00131 
00132     // Get the current extremes for this state
00133     Extremes extreme = extremes[state];
00134 
00135     // Longitudes in the CSV are negative for the US (New York is about -72,
00136     // California is about -122), so the farther east you go, the bigger the number.
00137     // Easternmost has the biggest longitude
00138     if(longitude > extreme.eastLongitude){
00139         extreme.east = zipCode;
00140         extreme.eastLongitude = longitude;
00141     }
00142     else if(longitude == extreme.eastLongitude && zipCode < extreme.east){
00143         extreme.east = zipCode;
00144     }
00145 
00146     // Westernmost has the smallest longitude
00147     if(longitude < extreme.westLongitude){
00148         extreme.west = zipCode;
00149         extreme.westLongitude = longitude;
00150     }
00151     else if(longitude == extreme.westLongitude && zipCode < extreme.west){
00152         extreme.west = zipCode;
00153     }
00154 
00155     // Northernmost has the biggest latitude
00156     if(latitude > extreme.northLatitude){
00157         extreme.north = zipCode;
00158         extreme.northLatitude = latitude;
00159     }
00160     else if(latitude == extreme.northLatitude && zipCode < extreme.north){
00161         extreme.north = zipCode;
00162     }
00163 
00164     // Southernmost has the smallest latitude
00165     if(latitude < extreme.southLatitude){
00166         extreme.south = zipCode;
00167         extreme.southLatitude = latitude;
00168     }
00169     else if(latitude == extreme.southLatitude && zipCode < extreme.south){
00170         extreme.south = zipCode;
00171     }
00172 
00173     // Save the updated extremes back into the map
00174     extremes[state] = extreme;
00175 }
```

References Extremes::east, Extremes::eastLongitude, extremes, latitude, longitude, Extremes::north, Extremes::northLatitude, Extremes::south, Extremes::southLatitude, state, Extremes::west, Extremes::westLongitude, and zipCode.

Referenced by readCSV().

3.1.3.3 getEasternmostZipCode()

```
int Buffer::getEasternmostZipCode (
            string state)
```

Returns EasternmostZipCode.

Parameters

state	State abbreviation already present in the extrema map.

Precondition

The state has been accumulated by findExtrema().

Returns

ZIP code for the requested direction.

Note

An unknown state inserts a zero-initialized map entry.

Definition at line 177 of file ZipCodeBuffer.cpp.

```
00177                                                {
00178     return extremes[state].east;
00179 }
```

References extremes, and state.

Referenced by displayTable().

3.1.3.4 getLatitude()

```
double Buffer::getLatitude ()
```

Returns Latitude.

Returns

Stored value.

Definition at line 46 of file ZipCodeBuffer.cpp.

```
00046                           {
00047     return latitude;
00048 }
```

References latitude.

3.1.3.5 getLongitude()

```
double Buffer::getLongitude ()
```

Returns Longitude.

Returns

Stored value.

Definition at line 50 of file ZipCodeBuffer.cpp.

```
00050                            {
00051     return longitude;
00052 }
```

References longitude.

3.1.3.6 getNorthernmostZipCode()

```
int Buffer::getNorthernmostZipCode (
            string state)
```

Returns NorthernmostZipCode.

Parameters

state	State abbreviation already present in the extrema map.

Precondition

The state has been accumulated by findExtrema().

Returns

ZIP code for the requested direction.

Note

An unknown state inserts a zero-initialized map entry.

Definition at line 185 of file ZipCodeBuffer.cpp.

```
00185                                                {
00186     return extremes[state].north;
00187 }
```

References extremes, and state.

Referenced by displayTable().

3.1.3.7 getPlaceName()

```
string Buffer::getPlaceName ()
```

Returns PlaceName.

Returns

Stored value.

Definition at line 42 of file ZipCodeBuffer.cpp.

```
00042                           {
00043     return placeName;
00044 }
```

References placeName.

3.1.3.8 getSouthernmostZipCode()

```
int Buffer::getSouthernmostZipCode (
            string state)
```

Returns SouthernmostZipCode.

Parameters

state	State abbreviation already present in the extrema map.

Precondition

The state has been accumulated by findExtrema().

Returns

ZIP code for the requested direction.

Note

An unknown state inserts a zero-initialized map entry.

Definition at line 189 of file ZipCodeBuffer.cpp.

```
00189                                                {
00190     return extremes[state].south;
00191 }
```

References extremes, and state.

Referenced by displayTable().

3.1.3.9 getState()

```
string Buffer::getState ()
```

Returns State.

Returns

Stored value.

Definition at line 38 of file ZipCodeBuffer.cpp.

```
00038                       {
00039     return state;
00040 }
```

References state.

3.1.3.10 getWesternmostZipCode()

```
int Buffer::getWesternmostZipCode (
            string state)
```

Returns WesternmostZipCode.

Parameters

state	State abbreviation already present in the extrema map.

Precondition

The state has been accumulated by findExtrema().

Returns

ZIP code for the requested direction.

Note

An unknown state inserts a zero-initialized map entry.

Definition at line 181 of file ZipCodeBuffer.cpp.

```
00181                                                {
00182     return extremes[state].west;
00183 }
```

References extremes, and state.

Referenced by displayTable().

3.1.3.11 getZipCode()

```
int Buffer::getZipCode ()
```

Returns ZipCode.

Returns

Stored value.

Definition at line 54 of file ZipCodeBuffer.cpp.

```
00054                       {
00055     return zipCode;
00056 }
```

References zipCode.

3.1.3.12 readCSV()

```
bool Buffer::readCSV (
            string fileName)
```

Reads the CSV and accumulates one record at a time.

Parameters

fileName	Path to a CSV with a header and columns ZIP, place, state, county, latitude, longitude.

Returns

false if opening fails; true after processing the file.

Skips records with empty ZIP, state, latitude, or longitude. Existing extrema remain when called again. Comma splitting does not handle quoted commas.

Exceptions

std::invalid_argument	If a numeric field cannot be converted.
std::out_of_range	If a numeric field is outside the conversion range.

Definition at line 62 of file ZipCodeBuffer.cpp.

```
00062                                   {
00063     ifstream file(fileName);
00064     if(!file.is_open()){
00065         return false;
00066     }
00067
00068     string line;
00069
00070     // Skip the header line
00071     getline(file, line);
00072
00073     while(getline(file, line)){
00074         stringstream ss(line);
00075
00076         // The columns in the file are: Zip Code, Place Name, State, County, Lat, Long
00077         string zip;
00078         string place;
00079         string state;
00080         string county;
00081         string lat;
00082         string lon;
00083
00084         getline(ss, zip, ',');
00085         getline(ss, place, ',');
00086         getline(ss, state, ',');
00087         getline(ss, county, ',');
00088         getline(ss, lat, ',');
00089         getline(ss, lon, ',');
00090
00091         // skip any line that is missing data
00092         if(zip == "" || state == "" || lat == "" || lon == ""){
00093             continue;
00094         }
00095
00096         // stoi turns a string into an int, stod turns a string into a double
00097         setZipCode(stoi(zip));
00098         setPlaceName(place);
00099         setState(state);
00100         setLatitude(stod(lat));
```

```
00101         setLongitude(stod(lon));
00102
00103         findExtrema();
00104     }
00105
00106     file.close();
00107     return true;
00108 }
```

References findExtrema(), setLatitude(), setLongitude(), setPlaceName(), setState(), setZipCode(), and state.

Referenced by main().

3.1.3.13 setLatitude()

```
void Buffer::setLatitude (
             double latitude)
```

Sets the current record Latitude.

Parameters

latitude	Value to store in the record buffer.

Definition at line 26 of file ZipCodeBuffer.cpp.

```
00026                                       {
00027     this->latitude = latitude;
00028 }
```

References latitude.

Referenced by readCSV().

3.1.3.14 setLongitude()

```
void Buffer::setLongitude (
             double longitude)
```

Sets the current record Longitude.

Parameters

longitude	Value to store in the record buffer.

Definition at line 30 of file ZipCodeBuffer.cpp.

```
00030                                        {
00031     this->longitude = longitude;
00032 }
```

References longitude.

Referenced by readCSV().

3.1.3.15 setPlaceName()

```
void Buffer::setPlaceName (
            string placeName)
```

Sets the current record PlaceName.

Parameters

placeName	Value to store in the record buffer.

Definition at line 22 of file ZipCodeBuffer.cpp.

```
00022                                           {
00023     this->placeName = placeName;
00024 }
```

References placeName.

Referenced by readCSV().

3.1.3.16 setState()

```
void Buffer::setState (
            string state)
```

Sets the current record State.

Parameters

state	Value to store in the record buffer.

Definition at line 18 of file ZipCodeBuffer.cpp.

```
00018                                 {
00019     this->state = state;
00020 }
```

References state.

Referenced by readCSV().

3.1.3.17 setZipCode()

```
void Buffer::setZipCode (
            int zipCode)
```

Sets the current record ZipCode.

Parameters

zipCode	Value to store in the record buffer.

Definition at line 34 of file ZipCodeBuffer.cpp.

```
00034                                   {
00035     this->zipCode = zipCode;
00036 }
```

References zipCode.

Referenced by readCSV().

3.1.4 Member Data Documentation

3.1.4.1 extremes

```
map<string, Extremes> Buffer::extremes  [private]
```

Definition at line 147 of file ZipCodeBuffer.h.

Referenced by displayTable(), findExtrema(), getEasternmostZipCode(), getNorthernmostZipCode(), getSouthernmostZipCode(), and getWesternmostZipCode().

3.1.4.2 latitude

```
double Buffer::latitude  [private]
```

Current record latitude.

Definition at line 143 of file ZipCodeBuffer.h.

Referenced by Buffer(), findExtrema(), getLatitude(), and setLatitude().

3.1.4.3 longitude

```
double Buffer::longitude  [private]
```

Current record longitude.

Definition at line 144 of file ZipCodeBuffer.h.

Referenced by Buffer(), findExtrema(), getLongitude(), and setLongitude().

3.1.4.4 placeName

```
string Buffer::placeName  [private]
```

Current record placeName.

Definition at line 142 of file ZipCodeBuffer.h.

Referenced by Buffer(), getPlaceName(), and setPlaceName().

3.1.4.5 state

```
string Buffer::state  [private]
```

Current record state.

Definition at line 141 of file ZipCodeBuffer.h.

Referenced by Buffer(), displayTable(), findExtrema(), getEasternmostZipCode(), getNorthernmostZipCode(), getSouthernmostZipCode(), getState(), getWesternmostZipCode(), readCSV(), and setState().

3.1.4.6 zipCode

```
int Buffer::zipCode  [private]
```

Current record zipCode.

Definition at line 145 of file ZipCodeBuffer.h.

Referenced by Buffer(), findExtrema(), getZipCode(), and setZipCode().

The documentation for this class was generated from the following files:

- ZipCodeBuffer.h
- ZipCodeBuffer.cpp

3.2 Extremes Struct Reference

ZIP codes and comparison coordinates for one state.

```
#include <ZipCodeBuffer.h>
```

Public Attributes

- int east
  Stored east value.
- int west
  Stored west value.
- int north
  Stored north value.
- int south
  Stored south value.
- double eastLongitude
  Stored eastLongitude value.
- double westLongitude
  Stored westLongitude value.
- double northLatitude
  Stored northLatitude value.
- double southLatitude
  Stored southLatitude value.

3.2.1 Detailed Description

ZIP codes and comparison coordinates for one state.

Definition at line 13 of file ZipCodeBuffer.h.

3.2.2 Member Data Documentation

3.2.2.1 east

```
int Extremes::east
```

Stored east value.

Definition at line 14 of file ZipCodeBuffer.h.

Referenced by Buffer::findExtrema().

3.2.2.2 eastLongitude

```
double Extremes::eastLongitude
```

Stored eastLongitude value.

Definition at line 19 of file ZipCodeBuffer.h.

Referenced by Buffer::findExtrema().

3.2.2.3 north

`int Extremes::north`

Stored north value.

Definition at line 16 of file ZipCodeBuffer.h.

Referenced by Buffer::findExtrema().

3.2.2.4 northLatitude

`double Extremes::northLatitude`

Stored northLatitude value.

Definition at line 21 of file ZipCodeBuffer.h.

Referenced by Buffer::findExtrema().

3.2.2.5 south

`int Extremes::south`

Stored south value.

Definition at line 17 of file ZipCodeBuffer.h.

Referenced by Buffer::findExtrema().

3.2.2.6 southLatitude

`double Extremes::southLatitude`

Stored southLatitude value.

Definition at line 22 of file ZipCodeBuffer.h.

Referenced by Buffer::findExtrema().

3.2.2.7 west

`int Extremes::west`

Stored west value.

Definition at line 15 of file ZipCodeBuffer.h.

Referenced by Buffer::findExtrema().

3.2.2.8 westLongitude

`double Extremes::westLongitude`

Stored westLongitude value.

Definition at line 20 of file ZipCodeBuffer.h.

Referenced by Buffer::findExtrema().

The documentation for this struct was generated from the following file:

- ZipCodeBuffer.h

Chapter 4

File Documentation

4.1 main.cpp File Reference

Application entry point for the ZIP code group project.

```
#include "ZipCodeBuffer.h"
```

Functions

- int main ()

  Runs the ZIP code table application.

4.1.1 Detailed Description

Application entry point for the ZIP code group project.

Definition in file main.cpp.

4.1.2 Function Documentation

4.1.2.1 main()

```
int main ()
```

Runs the ZIP code table application.

Returns

0 on success; 1 if the CSV cannot be opened.

Reads us_postal_codes.csv from the working directory and prints one row per state. Numeric conversion exceptions are not caught here.

Definition at line 11 of file main.cpp.

```
00011           {
00012     string fileName = "us_postal_codes.csv";
00013
00014     Buffer buffer;
00015
00016     // Read the file and find the farthest Zip Codes for each state
00017     if(!buffer.readCSV(fileName)){
00018         cout « "Error: could not open " « fileName « endl;
00019         return 1;
00020     }
00021
00022     // Print the table
00023     buffer.displayTable();
00024
00025     return 0;
00026 }
```

References Buffer::displayTable(), main(), and Buffer::readCSV().

Referenced by main().

4.2 main.cpp

Go to the documentation of this file.

```
00001 /** @file main.cpp
00002  * @brief Application entry point for the ZIP code group project.
00003  */
00004 #include "ZipCodeBuffer.h"
00005
00006 /** @brief Runs the ZIP code table application.
00007  * @return 0 on success; 1 if the CSV cannot be opened.
00008  * @details Reads us_postal_codes.csv from the working directory and prints
00009  * one row per state. Numeric conversion exceptions are not caught here.
00010  */
00011 int main(){
00012     string fileName = "us_postal_codes.csv";
00013
00014     Buffer buffer;
00015
00016     // Read the file and find the farthest Zip Codes for each state
00017     if(!buffer.readCSV(fileName)){
00018         cout « "Error: could not open " « fileName « endl;
00019         return 1;
00020     }
00021
00022     // Print the table
00023     buffer.displayTable();
00024
00025     return 0;
00026 }
```

4.3 ZipCodeBuffer.cpp File Reference

Implements CSV parsing, extrema accumulation, and table output.

```
#include "ZipCodeBuffer.h"
#include <iomanip>
#include <sstream>
```

4.3.1 Detailed Description

Implements CSV parsing, extrema accumulation, and table output.

Function contracts are documented in ZipCodeBuffer.h.

Definition in file ZipCodeBuffer.cpp.

4.4 ZipCodeBuffer.cpp

Go to the documentation of this file.

```
00001 /** @file ZipCodeBuffer.cpp
00002  * @brief Implements CSV parsing, extrema accumulation, and table output.
00003  * @details Function contracts are documented in ZipCodeBuffer.h.
00004  */
00005 #include "ZipCodeBuffer.h"
00006 #include <iomanip>
00007 #include <sstream>
00008
00009
00010 Buffer::Buffer(){
00011     state = "";
00012     placeName = "";
00013     latitude = 0.0;
00014     longitude = 0.0;
00015     zipCode = 0;
00016 }
00017
00018 void Buffer::setState(string state){
00019     this->state = state;
00020 }
00021
00022 void Buffer::setPlaceName(string placeName){
00023     this->placeName = placeName;
00024 }
00025
00026 void Buffer::setLatitude(double latitude){
00027     this->latitude = latitude;
00028 }
00029
00030 void Buffer::setLongitude(double longitude){
00031     this->longitude = longitude;
00032 }
00033
00034 void Buffer::setZipCode(int zipCode){
00035     this->zipCode = zipCode;
00036 }
00037
00038 string Buffer::getState(){
00039     return state;
00040 }
00041
00042 string Buffer::getPlaceName(){
00043     return placeName;
00044 }
00045
00046 double Buffer::getLatitude(){
00047     return latitude;
00048 }
00049
00050 double Buffer::getLongitude(){
00051     return longitude;
00052 }
00053
00054 int Buffer::getZipCode(){
00055     return zipCode;
00056 }
00057
00058
00059 // Reads the CSV file one line at a time.
00060 // Each line is split into its fields, stored in the buffer,
```

```
00061 // and then checked against the extremes for its state.
00062 bool Buffer::readCSV(string fileName){
00063     ifstream file(fileName);
00064     if(!file.is_open()){
00065         return false;
00066     }
00067
00068     string line;
00069
00070     // Skip the header line
00071     getline(file, line);
00072
00073     while(getline(file, line)){
00074         stringstream ss(line);
00075
00076         // The columns in the file are: Zip Code, Place Name, State, County, Lat, Long
00077         string zip;
00078         string place;
00079         string state;
00080         string county;
00081         string lat;
00082         string lon;
00083
00084         getline(ss, zip, ',');
00085         getline(ss, place, ',');
00086         getline(ss, state, ',');
00087         getline(ss, county, ',');
00088         getline(ss, lat, ',');
00089         getline(ss, lon, ',');
00090
00091         // skip any line that is missing data
00092         if(zip == "" || state == "" || lat == "" || lon == ""){
00093             continue;
00094         }
00095
00096         // stoi turns a string into an int, stod turns a string into a double
00097         setZipCode(stoi(zip));
00098         setPlaceName(place);
00099         setState(state);
00100         setLatitude(stod(lat));
00101         setLongitude(stod(lon));
00102
00103         findExtrema();
00104     }
00105
00106     file.close();
00107     return true;
00108 }
00109
00110 // Checks the record in the buffer against the farthest Zip Codes
00111 // found so far for its state, and replaces any that it beats.
00112 // If two Zip Codes are tied, the smaller Zip Code is kept so the
00113 // answer is the same no matter what order the file is in.
00114 void Buffer::findExtrema(){
00115
00116     // First time seeing this state, so this Zip Code is the farthest in every direction
00117     if(extremes.count(state) == 0){
00118         Extremes extreme;
00119         extreme.east = zipCode;
00120         extreme.west = zipCode;
00121         extreme.north = zipCode;
00122         extreme.south = zipCode;
00123         extreme.eastLongitude = longitude;
00124         extreme.westLongitude = longitude;
00125         extreme.northLatitude = latitude;
00126         extreme.southLatitude = latitude;
00127
00128         extremes[state] = extreme;
00129         return;
00130     }
00131
00132     // Get the current extremes for this state
00133     Extremes extreme = extremes[state];
00134
00135     // Longitudes in the CSV are negative for the US (New York is about -72,
00136     // California is about -122), so the farther east you go, the bigger the number.
00137     // Easternmost has the biggest longitude
00138     if(longitude > extreme.eastLongitude){
00139         extreme.east = zipCode;
00140         extreme.eastLongitude = longitude;
00141     }
```

```
00142     else if(longitude == extreme.eastLongitude && zipCode < extreme.east){
00143         extreme.east = zipCode;
00144     }
00145
00146     // Westernmost has the smallest longitude
00147     if(longitude < extreme.westLongitude){
00148         extreme.west = zipCode;
00149         extreme.westLongitude = longitude;
00150     }
00151     else if(longitude == extreme.westLongitude && zipCode < extreme.west){
00152         extreme.west = zipCode;
00153     }
00154
00155     // Northernmost has the biggest latitude
00156     if(latitude > extreme.northLatitude){
00157         extreme.north = zipCode;
00158         extreme.northLatitude = latitude;
00159     }
00160     else if(latitude == extreme.northLatitude && zipCode < extreme.north){
00161         extreme.north = zipCode;
00162     }
00163
00164     // Southernmost has the smallest latitude
00165     if(latitude < extreme.southLatitude){
00166         extreme.south = zipCode;
00167         extreme.southLatitude = latitude;
00168     }
00169     else if(latitude == extreme.southLatitude && zipCode < extreme.south){
00170         extreme.south = zipCode;
00171     }
00172
00173     // Save the updated extremes back into the map
00174     extremes[state] = extreme;
00175 }
00176
00177 int Buffer::getEasternmostZipCode(string state){
00178     return extremes[state].east;
00179 }
00180
00181 int Buffer::getWesternmostZipCode(string state){
00182     return extremes[state].west;
00183 }
00184
00185 int Buffer::getNorthernmostZipCode(string state){
00186     return extremes[state].north;
00187 }
00188
00189 int Buffer::getSouthernmostZipCode(string state){
00190     return extremes[state].south;
00191 }
00192
00193 // Prints a header row and then one row for each state.
00194 // The map already keeps the states in alphabetical order.
00195 void Buffer::displayTable(){
00196     cout « left;
00197     cout « setw(7) « "State";
00198     cout « setw(14) « "Easternmost";
00199     cout « setw(14) « "Westernmost";
00200     cout « setw(14) « "Northernmost";
00201     cout « setw(14) « "Southernmost" « endl;
00202     cout « "-------------------------------------------------------------" « endl;
00203
00204     // Go through every state in the map
00205     for(pair<string, Extremes> entry : extremes){
00206         string state = entry.first;
00207
00208         cout « left « setfill(' ') « setw(7) « state;
00209
00210         // setw(5) and setfill('0') print Zip Codes like 501 as 00501
00211         cout « right « setfill('0');
00212         cout « setw(5) « getEasternmostZipCode(state) « "         ";
00213         cout « setw(5) « getWesternmostZipCode(state) « "         ";
00214         cout « setw(5) « getNorthernmostZipCode(state) « "          ";
00215         cout « setw(5) « getSouthernmostZipCode(state) « endl;
00216     }
00217
00218     cout « setfill(' ');
00219 }
```

4.5 ZipCodeBuffer.h File Reference

Record buffer and per-state geographic extrema declarations.

```
#include <fstream>
#include <iostream>
#include <map>
#include <string>
```

Classes

- struct Extremes

  ZIP codes and comparison coordinates for one state.
- class Buffer

  Reads records sequentially and accumulates geographic extrema.

4.5.1 Detailed Description

Record buffer and per-state geographic extrema declarations.

Definition in file ZipCodeBuffer.h.

4.6 ZipCodeBuffer.h

Go to the documentation of this file.

```
00001 /** @file ZipCodeBuffer.h
00002  * @brief Record buffer and per-state geographic extrema declarations.
00003  */
00004 #ifndef ZIPCODEBUFFER_H
00005 #define ZIPCODEBUFFER_H
00006 #include <fstream>
00007 #include <iostream>
00008 #include <map>
00009 #include <string>
00010 using namespace std;
00011
00012 /** @brief ZIP codes and comparison coordinates for one state. */
00013 struct Extremes{
00014     int east; ///< Stored east value.
00015     int west; ///< Stored west value.
00016     int north; ///< Stored north value.
00017     int south; ///< Stored south value.
00018
00019     double eastLongitude; ///< Stored eastLongitude value.
00020     double westLongitude; ///< Stored westLongitude value.
00021     double northLatitude; ///< Stored northLatitude value.
00022     double southLatitude; ///< Stored southLatitude value.
00023 };
00024
00025 /** @brief Reads records sequentially and accumulates geographic extrema.
00026  * @details Stores the current record and a state-keyed map. Equal coordinates
00027  * select the smaller ZIP code, making results independent of record order.
00028  */
00029 class Buffer{
00030 public:
00031
```

```
00032     /** @brief Initializes an empty record and empty extrema map. */
00033     Buffer();
00034
00035     /** @brief Sets the current record State.
00036      * @param state Value to store in the record buffer.
00037      */
00038     void setState(string state);
00039
00040     /** @brief Sets the current record PlaceName.
00041      * @param placeName Value to store in the record buffer.
00042      */
00043     void setPlaceName(string placeName);
00044
00045     /** @brief Sets the current record Latitude.
00046      * @param latitude Value to store in the record buffer.
00047      */
00048     void setLatitude(double latitude);
00049
00050     /** @brief Sets the current record Longitude.
00051      * @param longitude Value to store in the record buffer.
00052      */
00053     void setLongitude(double longitude);
00054
00055     /** @brief Sets the current record ZipCode.
00056      * @param zipCode Value to store in the record buffer.
00057      */
00058     void setZipCode(int zipCode);
00059
00060     /** @brief Returns State.
00061      * @return Stored value.
00062      */
00063     string getState();
00064
00065     /** @brief Returns PlaceName.
00066      * @return Stored value.
00067      */
00068     string getPlaceName();
00069
00070     /** @brief Returns Latitude.
00071      * @return Stored value.
00072      */
00073     double getLatitude();
00074
00075     /** @brief Returns Longitude.
00076      * @return Stored value.
00077      */
00078     double getLongitude();
00079
00080     /** @brief Returns ZipCode.
00081      * @return Stored value.
00082      */
00083     int getZipCode();
00084
00085     /** @brief Reads the CSV and accumulates one record at a time.
00086      * @param fileName Path to a CSV with a header and columns ZIP, place, state, county, latitude,
      longitude.
00087      * @return false if opening fails; true after processing the file.
00088      * @details Skips records with empty ZIP, state, latitude, or longitude.
00089      * Existing extrema remain when called again. Comma splitting does not handle quoted commas.
00090      * @throws std::invalid_argument If a numeric field cannot be converted.
00091      * @throws std::out_of_range If a numeric field is outside the conversion range.
00092      */
00093     bool readCSV(string fileName);
00094
00095     /** @brief Updates state extrema using the current buffered record.
00096      * @details East uses maximum longitude, west minimum longitude, north
00097      * maximum latitude, and south minimum latitude. Ties use the smaller ZIP.
00098      */
00099     void findExtrema();
00100
00101     /** @brief Returns EasternmostZipCode.
00102      * @param state State abbreviation already present in the extrema map.
00103      * @pre The state has been accumulated by findExtrema().
00104      * @return ZIP code for the requested direction.
00105      * @note An unknown state inserts a zero-initialized map entry.
00106      */
00107     int getEasternmostZipCode(string state);
00108
00109     /** @brief Returns WesternmostZipCode.
00110      * @param state State abbreviation already present in the extrema map.
00111      * @pre The state has been accumulated by findExtrema().
```

```
00112      * @return ZIP code for the requested direction.
00113      * @note An unknown state inserts a zero-initialized map entry.
00114      */
00115     int getWesternmostZipCode(string state);
00116
00117     /** @brief Returns NorthernmostZipCode.
00118      * @param state State abbreviation already present in the extrema map.
00119      * @pre The state has been accumulated by findExtrema().
00120      * @return ZIP code for the requested direction.
00121      * @note An unknown state inserts a zero-initialized map entry.
00122      */
00123     int getNorthernmostZipCode(string state);
00124
00125     /** @brief Returns SouthernmostZipCode.
00126      * @param state State abbreviation already present in the extrema map.
00127      * @pre The state has been accumulated by findExtrema().
00128      * @return ZIP code for the requested direction.
00129      * @note An unknown state inserts a zero-initialized map entry.
00130      */
00131     int getSouthernmostZipCode(string state);
00132
00133     /** @brief Prints the state extrema table to standard output.
00134      * @details Rows follow alphabetical state order. Columns are state, east,
00135      * west, north, south. ZIP codes are padded to five digits.
00136      */
00137     void displayTable();
00138
00139 private:
00140
00141     string state; ///< Current record state.
00142     string placeName; ///< Current record placeName.
00143     double latitude; ///< Current record latitude.
00144     double longitude; ///< Current record longitude.
00145     int zipCode; ///< Current record zipCode.
00146
00147     map<string, Extremes> extremes; // one entry per state, a map keeps them in alphabetical order
00148
00149 };
00150
00151 #endif
```

Index

Buffer, 5
    Buffer, 6
    displayTable, 7
    extremes, 16
    findExtrema, 7
    getEasternmostZipCode, 8
    getLatitude, 9
    getLongitude, 9
    getNorthernmostZipCode, 9
    getPlaceName, 10
    getSouthernmostZipCode, 10
    getState, 11
    getWesternmostZipCode, 11
    getZipCode, 12
    latitude, 16
    longitude, 16
    placeName, 16
    readCSV, 12
    setLatitude, 14
    setLongitude, 14
    setPlaceName, 14
    setState, 15
    setZipCode, 15
    state, 17
    zipCode, 17

displayTable
    Buffer, 7

east
    Extremes, 18
eastLongitude
    Extremes, 18
Extremes, 17
    east, 18
    eastLongitude, 18
    north, 18
    northLatitude, 19
    south, 19
    southLatitude, 19
    west, 19
    westLongitude, 19
extremes
    Buffer, 16

findExtrema
    Buffer, 7

getEasternmostZipCode
    Buffer, 8
getLatitude
    Buffer, 9
getLongitude
    Buffer, 9
getNorthernmostZipCode
    Buffer, 9
getPlaceName
    Buffer, 10
getSouthernmostZipCode
    Buffer, 10
getState
    Buffer, 11
getWesternmostZipCode
    Buffer, 11
getZipCode
    Buffer, 12

latitude
    Buffer, 16
longitude
    Buffer, 16

main
    main.cpp, 21
main.cpp, 21
    main, 21

north
    Extremes, 18
northLatitude
    Extremes, 19

placeName
    Buffer, 16

readCSV
    Buffer, 12

setLatitude
    Buffer, 14
setLongitude
    Buffer, 14
setPlaceName

Buffer, 14
setState
Buffer, 15
setZipCode
Buffer, 15
south
Extremes, 19
southLatitude
Extremes, 19
state
Buffer, 17

west
Extremes, 19
westLongitude
Extremes, 19

zipCode
Buffer, 17
ZipCodeBuffer.cpp, 22
ZipCodeBuffer.h, 26